PROPOSAL TUGAS AKHIR – ET234801

IMPLEMENTASI DAN EVALUASI NETWORK EXPERIENCE BLACK-BOX "MICRO-UXI" BERBASIS ARDUINO UNO Q MENGGUNAKAN PENDEKATAN EVENT-DRIVEN MONITORING

MUHAMMAD FAISHAL RIZQY

NRP 5027221026

Dosen Pembimbing

Hatma Suryotrisongko, S.Kom., M.Eng., Ph.D.

NIP 198405182014041001

Hafara Firdausi, S.Kom., M.Kom.

NIP 2022199812030

Program Studi S1 Teknologi Informasi

Departemen Teknologi Informasi

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

LEMBAR PENGESAHAN

IMPLEMENTASI DAN EVALUASI NETWORK EXPERIENCE BLACK-BOX "MICRO-UXI" BERBASIS ARDUINO UNO Q MENGGUNAKAN PENDEKATAN EVENT-DRIVEN MONITORING

PROPOSAL TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat

memperoleh gelar Sarjana Komputer pada

Program Studi S-1 Teknologi Informasi

Departemen Teknologi Informasi

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Oleh : **MUHAMMAD FAISHAL RIZQY**

NRP. 5027221026

Disetujui oleh Tim Penguji Proposal Tugas Akhir :

1.	Hatma Suryotrisongko, S.Kom., M.Eng., Ph.D.	Pembimbing
2.	Hafara Firdausi, S.Kom., M.Kom.	Ko-pembimbing
3.	Nama dan gelar penguji	Penguji
4.	Nama dan gelar penguji	Penguji
5.	Nama dan gelar pengujim	Penguji

SURABAYA

Februari, 2026

APPROVAL SHEET

PERNYATAAN ORISINALITAS

STATEMENT OF ORIGINALITY

ABSTRAK

IMPLEMENTASI DAN EVALUASI NETWORK EXPERIENCE BLACK-BOX "MICRO-UXI" BERBASIS ARDUINO UNO Q MENGGUNAKAN PENDEKATAN EVENT-DRIVEN MONITORING

Nama Mahasiswa / NRP	**:**	**Muhammad Faishal Rizqy / 502721026**
Departemen	**:**	**Teknologi Informasi FTEIC - ITS**
Dosen Pembimbing 1	**:**	**Hatma Suryotrisongko, S.Kom., M.Eng., Ph.D.**
Dosen Pembimbing 1	**:**	**Hafara Firdausi, S.Kom., M.Kom.**

Abstrak

Pemantauan infrastruktur jaringan modern tidak cukup hanya memastikan ketersediaan layanan, tetapi juga harus menangkap kualitas pengalaman pengguna (*Quality of Experience*) secara proaktif; namun, solusi komersial seperti Aruba UXI cenderung mahal dan berorientasi pada skala enterprise sehingga kurang sesuai untuk lingkungan kecil–menengah. Proposal ini mengusulkan Micro-UXI, sebuah sistem monitoring berbasis perangkat edge berbiaya rendah yang berperan sebagai network experience black-box event recorder dengan mekanisme event-trigger, yakni merekam detail hanya ketika terindikasi degradasi/anomali agar lebih efisien daripada pemantauan periodik. Implementasi diarahkan pada Arduino Uno Q dengan fungsi paralel: pengukuran telemetri pengalaman jaringan secara kontinu (misalnya status Wi-Fi, RTT/loss, DNS, dan HTTP *timing*) serta perekaman *evidence bundl*e terstruktur (*pre-event, event, post-event*) yang dilengkapi *snapshot* diagnostik jaringan. Kinerja deteksi *event-driven* dievaluasi melalui eksperimen gangguan terkontrol (*fault injection*) dengan ground truth, lalu dibandingkan terhadap baseline monitoring periodik berbasis ambang statis menggunakan metrik *precision/recall/F1*, *false alarm rate*, dan *mean time to detect*, serta dinilai kelengkapan bukti melalui *Reconstruction Completeness Score*. Analisis *overhead* (CPU, RAM, *storage*, *bandwidth*) pada mode normal dan mode event diharapkan menghasilkan rekomendasi kelayakan desain yang ekonomis, modular, dan adaptif untuk memperkuat investigasi insiden pada perangkat dengan sumber daya terbatas.

Kata kunci: Quality of Experience (QoE), monitoring jaringan, event-driven monitoring, perangkat edge, black-box event recorder, Arduino UNO R4, deteksi anomali, fault injection

ABSTRACT

IMPLEMENTATION AND EVALUATION OF NETWORK EXPERIENCE BLACKBOX ON ARDUINO UNO Q USING EVENT-DRIVEN MONITORING APPROACH

Nama Mahasiswa / NRP	**:**	**Muhammad Faishal Rizqy / 502721026**
Departemen	**:**	**Teknologi Informasi FTEIC - ITS**
Dosen Pembimbing 1	**:**	**Hatma Suryotrisongko, S.Kom., M.Eng., Ph.D.**
Dosen Pembimbing 1	**:**	**Hafara Firdausi, S.Kom., M.Kom.**

Abstract

Monitoring modern network infrastructure is no longer limited to service availability; it must also capture user-perceived experience (Quality of Experience) proactively. However, commercial platforms such as Aruba UXI are often costly and optimized for enterprise-scale deployments, making them less suitable for small-to-medium environments. This proposal introduces Micro-UXI, a low-cost edge-based monitoring system designed as a network experience black-box event recorder with an event-trigger mechanism that records detailed diagnostics only when degradation or anomalies are indicated, improving efficiency over purely periodic monitoring. The system is implemented on an Arduino Uno Q and operates in parallel modes: continuous experience telemetry collection (e.g., Wi-Fi status, RTT/loss, DNS, and HTTP timing) and structured evidence bundle capture (pre-event, event, post-event) enriched with network diagnostic snapshots. Event-driven detection performance is assessed through controlled impairment experiments (fault injection) with ground truth, then benchmarked against a static-threshold periodic monitoring baseline using precision/recall/F1, false-alarm rate, and mean time to detect, while evidence completeness is quantified via a Reconstruction Completeness Score. The expected outcome is a feasibility and design recommendation for an economical, modular, and adaptive monitoring approach that strengthens incident investigation on resource-constrained devices.

Keywords: ***Quality of Experience (QoE), network monitoring, event-driven monitoring, edge device, black-box event recorder, Arduino UNO R4, anomaly detection, fault injection***

DAFTAR ISI

BAB 1 PENDAHULUAN 12
1.1 Latar Belakang 12
1.2 Rumusan Masalah 12
1.3 Batasan Masalah 13
1.4 Tujuan 13
1.5 Manfaat 14
BAB 2 TINJAUAN PUSTAKA 15
2.1 Hasil Penelitian Terdahulu 15
2.2 Dasar Teori 16
2.2.1 Quality of experience (QoE) dan digital experience monitoring 16
2.2.2 Active/synthetic probing sebagai monitoring 17
2.2.3 Event-driven / event-triggered monitoring 17
2.2.4 Ring buffer dan pre/post window untuk perekaman bukti insiden 17
2.2.5 Fault injection dan ground truth untuk evaluasi sistem deteksi 18
BAB 3 METODOLOGI 19
3.1 Metode yang digunakan 19
3.1.1 Tahapan penelitian 19
3.1.2 Data yang diguanakan 20
3.2 Bahan dan peralatan yang digunakan 21
3.2.1 Perangkat keras 21
3.2.2 Perangkat lunak 21
3.3 Perancangan arsitektur sistem 22
3.3.1 Arsitektur fungsional sistem 22
3.3.2 Alur kerja sistem dan model keadaan 22
3.3.3 Definisi event dan parameter awal deteksi 23
3.3.4 Desain baseline dan aturan trigger 23
3.3.5 Desain evidence bundle 23
3.4 Implementasi prototipe 24
3.4.1 Implementasi pada sensor 24
3.4.2 implementasi pada server 24
3.5 Verifikasi fungsional sistem 25
3.6 Evaluasi eksperimental dan pembandingan 25
3.6.1 Desain run dan unit analisis 25
3.6.2 Lingkungan uji dan kontrol variabel 26
3.6.3 Skenario uji (fault injection) 26
3.6.4 Baseline pembanding 26
3.6.5 Prosedur uji per run 26
3.6.6 Pencocokan event-fault (ground truth alignment) 27
3.6.7 Pengukuran overhead 27

3.7 Analisis data dan pelaporan 27
3.7.1 Pra-pemrosesan data 27
3.7.2 Metrik evaluasi 28
3.8 Timeline penelitian 28
BAB 4 HASIL DAN PEMBAHASAN 29
BAB 5 KESIMPULAN DAN SARAN 31

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR SIMBOL

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan infrastruktur jaringan saat ini tidak hanya menuntut ketersediaan layanan (*availability*), tetapi juga kualitas pengalaman pengguna (*Quality of Experience*). Banyak organisasi telah mengadopsi solusi monitoring seperti Aruba UXI untuk melakukan pemantauan performa jaringan secara aktif dan kontinu. Jenis sistem seperti ini mampu melakukan pengukuran yang mencakup berbagai lapisan seperti performa aplikasi dan konektivitas jaringan, sehingga berbagai macam gangguan dapat terdeteksi secara proaktif sebelum terjadi insiden yang berdampak luas.

Akan tetapi, solusi komersial seperti Aruba UXI tersebut umumnya memiliki biaya implementasi yang relatif mahal dan dimaksudkan untuk skala *enterprise*. Hal ini menyebabkan kebutuhan akan metode alternatif yang lebih terjangkau dan dapat disesuaikan untuk lingkungan berskala kecil hingga menengah seperti laboratorium atau lembaga pendidikan.

Salah satu pendekatan relevan yang ditawarkan dalam penelitian ini adalah konsep Micro-UXI, yaitu sistem *monitoring* berbasis perangkat *edge* dengan biaya rendah yang berfungsi sebagai *Network Experience Black-Box "Event Recorder"*. Sistem ini dirancang untuk mencatat hanya apabila terjadi degradasi performa jaringan atau terjadinya suatu anomali tertentu berdasarkan mekanisme *event-trigger*. Pendekatan ini bertujuan untuk meningkatkan efisiensi penggunaan sumber daya dibandingkan dengan metode pemantauan periodik konvensional.

Tantangan utama muncul ketika arsitektur yang digunakan diimplementasikan pada perangkat dengan keterbatasan komputasi dan memori. Keterbatasan ini berpotensi memengaruhi ketepatan deteksi, latensi pencatatan, dan stabilitas sistem dalam mendeteksi adanya gangguan pada jaringan.

Oleh karena itu, penelitian ini berfokus pada perancangan dan implementasi Micro-UXI berbasis Arduino UNO Q sebagai *Network Experience Black-Box "Event Recorder"*. Studi ini bertujuan untuk mengevaluasi sejauh mana pendekatan *event-driven monitoring* dapat diimplementasikan secara efektif pada perangkat berbiaya rendah dengan keterbatasan sumber daya, serta menganalisis performa sistem dalam mendeteksi dan merekam kejadian degradasi jaringan secara terstruktur.

Dengan pendekatan tersebut, penelitian ini diharapkan dapat memberikan kontribusi terhadap pengembangan sistem monitoring jaringan yang lebih ekonomis, modular, dan adaptif terhadap berbagai kondisi infrastruktur.

1.2 Rumusan Masalah

1. Bagaimana merancang dan mengimplementasikan prototipe Micro-UXI berbasis Arduino Uno Q sebagai *network event recorder* yang melakukan pengukuran

pengalaman jaringan dan menghasilkan keluaran berupa metrik *monitoring* serta *evidence bundle* (*pre-event, event, post-event*)?

2. Seberapa baik kinerja deteksi *event* Micro-UXI (*event-driven detection & recording*) dibandingkan dengan *baseline monitoring* periodik berbasis *threshold* statis, ditinjau dari *precision/recall/F1, false alarm rate* (*FAR*), dan *latency* deteksi (*MTTD*) pada skenario gangguan terkontrol dengan *ground truth*?
3. Seberapa lengkap bukti (*evidence bundle*) yang dihasilkan Micro-UXI untuk mendukung rekonstruksi insiden, ditinjau dari *Reconstruction Completeness Score* (*RCS*)?
4. Bagaimana *overhead* sistem (CPU, RAM, *storage, bandwidth*) pada mode normal dan mode *event* saat Micro-UXI beroperasi secara kontinu?

1.3 Batasan Masalah

1. Sistem Micro-UXI diimplementasikan pada Arduino Uno Q (RAM 2 GB).
2. Konfigurasi SSID/AP/*channel* dan target uji (DNS/URL) dijaga tetap dalam satu run untuk mengurangi variabel pengganggu.
3. *Event* dibatasi pada *micro-events* degradasi pengalaman jaringan yang terukur dan dapat direplikasi.
4. Evaluasi kinerja deteksi dilakukan melalui skenario gangguan terkontrol (*fault injection*) dengan pencatatan *ground truth* (waktu mulai dan selesai gangguan).
5. Bukti yang dikumpulkan dibatasi pada *minimal evidence bundle* untuk mendukung rekonstruksi insiden, yaitu *pre-event window*, *event window*, *post-event window*, serta *snapshot* diagnostik (misalnya status koneksi Wi-Fi, konfigurasi IP, rute jaringan, dan *resolver* DNS) dan *timing* akses layanan (misalnya *curl timing*). Penelitian ini tidak menargetkan *full packet capture* (PCAP penuh) sebagaimana alat forensik jaringan seperti Wireshark.
6. Evaluasi dibatasi pada metrik yang sesuai dengan rumusan masalah, yaitu *Precision/Recall/F1*, *False Alarm Rate* (*FAR*), *Mean Time To Detect* (*MTTD*), *Reconstruction Completeness Score* (RCS), serta *overhead* sistem (CPU, RAM, *storage*, dan *bandwidth*).

1.4 Tujuan

1. Merancang dan mengimplementasikan prototipe Micro-UXI pada Arduino UNO Q yang mampu menghasilkan metrik *monitoring* dan *evidence bundle* (*pre-event, event, post-event*).
2. Mengevaluasi kinerja deteksi *event* Micro-UXI dibandingkan dengan *baseline monitoring* periodik berbasis *threshold* statis, menggunakan metrik *precision*, *recall*, F1-*score*, *false alarm rate* (FAR), serta *mean time to detect* (MTTD) pada skenario gangguan terkontrol dengan *ground truth*.
3. Mengevaluasi kelengkapan bukti rekonstruksi yang dihasilkan Micro-UXI melalui pengukuran *Reconstruction Completeness Score* (RCS).
4. Menganalisis *overhead* sistem selama operasi kontinu, mencakup penggunaan CPU, RAM, *storage*, dan *bandwidth*, baik pada kondisi normal maupun saat terjadi *event*, untuk menilai kelayakan implementasi pada perangkat berbiaya rendah dengan keterbatasan sumber daya.

1.5 Manfaat

1. Memberikan kontribusi pengetahuan mengenai implementasi *event-driven monitoring* berbiaya rendah dalam konteks *monitoring* jaringan.
2. Menyediakan kerangka evaluasi yang terukur untuk sistem *event recorder* jaringan melalui metrik deteksi, kelengkapan bukti, serta *overhead* sumber daya.
3. Menjadi landasan kelanjutan riset pada topik serupa.
4. Menyediakan prototipe solusi *monitoring* yang lebih terjangkau dan dapat diadaptasi untuk lingkungan berskala kecil-menengah, tanpa bergantung pada perangkat DEM komersial yang mahal.
5. Meningkatkan kemampuan investigasi gangguan jaringan melalui penyediaan *evidence bundle* yang terstruktur.
6. Memberikan rekomendasi teknis terkait konfigurasi dan kelayakan operasional Micro-UXI pada perangkat dengan keterbatasan sumber daya.

BAB 2 TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Tabel 2.1 Hasil penelitian terdahulu

No	Peneliti & Tahun	Fokus Penelitian	Temuan Utama	Relevansi Dengan Penelitian
1	Baswara, J. A. (2026). Implementasi platform open-source untuk monitoring User Experience Insight (UXI) jaringan nirkabel menggunakan perangkat Raspberry Pi	Implementasi UXI-Lite (sensor UXI low-cost) berbasis Raspberry Pi 4 dan evaluasi paritas pengukuran vs HPE Aruba UXI pada jaringan Wi-Fi.	Paritas terbaik pada ping latency (ρ tinggi; MAE rendah), namun HTTP GET dan throughput menunjukkan selisih lebih besar. UXI-Lite layak untuk indikasi degradasi konektivitas, namun belum ideal sebagai full replacement tanpa kalibrasi.	Merupakan dasar awal pengembangan sistem UXI-Lite dan dasar awal dari penelitian ini.
2	Miranda, G., Municio, E., Marquez-Barja, J. M., & Macedo, D. F. (2022). Machine Learning-based End-to-End QoE Monitoring Using Active Network Probing. IEEE ICIN 2022, pp. 40–47. DOI: 10.1109/ICIN53892.2022.9758123.	Monitoring QoE end-to-end menggunakan active probing dan pemodelan berbasis ML.	Memperlihatkan bahwa data hasil probing aktif dapat dipakai untuk mengestimasi/monitor QoE secara end-to-end.	Memperkuat landasan bahwa pengukuran aktif di sisi klien merupakan sinyal yang sah untuk menilai pengalaman jaringan.
3	Kučera, J., Popescu, D. A., Wang, H., Moore, A. W., Kořenek, J., & Antichi, G. (2020). Enabling Event-Triggered Data Plane Monitoring.	Monitoring jaringan event-triggered/push-based.	Pendekatan event-triggered menekan overhead dengan mengirim data/alert hanya saat event terdeteksi.	Menyediakan rujukan kuat bahwa arsitektur push/event-triggered memang dirancang untuk menurunkan overhead

	ACM SOSR ’20, pp. 14–26. DOI: 10.1145/3373360.3380830.			dibanding polling.
4	Zheng, H., Tian, C., Yang, T., Lin, H., Liu, C., Zhang, Z., Dou, W., & Chen, G. (2022). FlyMon: Enabling On-the-Fly Task Reconfiguration for Network Measurement. ACM SIGCOMM ’22, pp. 486–502. DOI: 10.1145/3544216.3544239.	Measurement dengan reconfigurable tasks untuk menyeimbangkan kebutuhan pengukuran vs resource terbatas.	Menunjukkan pentingnya mekanisme pemilihan/penyesuaian tugas monitoring agar tetap efektif namun tidak boros resource.	Mendukung argumen bahwa desain monitoring harus sadar keterbatasan sumber daya.
4	Zheng, H., Tian, C., Yang, T., Lin, H., Liu, C., Zhang, Z., Dou, W., & Chen, G. (2022). FlyMon: Enabling On-the-Fly Task Reconfiguration for Network Measurement. ACM SIGCOMM ’22, pp. 486–502. DOI: 10.1145/3544216.3544239.	Measurement dengan reconfigurable tasks untuk menyeimbangkan kebutuhan pengukuran vs resource terbatas.	Menunjukkan pentingnya mekanisme pemilihan/penyesuaian tugas monitoring agar tetap efektif namun tidak boros resource.	Mendukung argumen bahwa desain monitoring harus sadar keterbatasan sumber daya.

2.2 Dasar Teori

2.2.1 Quality of experience (QoE) dan digital experience monitoring

Pada sistem jaringan modern, kualitas layanan perlu dinilai dari segi kualitas pengalaman pengguna dalam mengakses layanan jaringan dan aplikasi. Konsep *quality of experience* (QoE) menjelaskan bahwa kualitas yang didapat oleh pengguna merupakan hasil kolektif dari beberapa faktor, sehingga adanya suatu degradasi pada satu atau lebih faktor dapat menyebabkan menurunnya kualitas pengalaman pengguna (Barakabitze et al., 2020).

Sejalan dengan hal tersebut, praktik *digital experience monitoring* mengarah pada pemantauan yang menempatkan pengalaman pengguna sebagai objek observasi. Pendekatan ini akhirnya menimbulkan kebutuhan deteksi gangguan yang proaktif sekaligus memberikan

konteks yang lebih dekat dengan pengalaman pengguna dibandingkan hanya dengan deteksi perangkat jaringan (Barakabitze et al., 2020).

2.2.2 Active/synthetic probing sebagai monitoring

Active probing atau *synthetic/active measurement* dilakukan dengan mengeksekusi paket uji atau transaksi uji secara terkontrol untuk mendapatkan indikator yang konsisten. Keunggulan dari pendekatan ini adalah ia dapat merepresentasikan pengalaman pengguna tanpa harus mengakses *payload traffic* pengguna yang bersifat sensitif.

Miranda et al. (2022) menunjukkan bahwa sinyal probing aktif dapat digunakan untuk melacak dan mengukur kualitas *end-to-end* pada berbagai kondisi konektivitas, mendukung pemantauan pengalaman dengan perspektif yang dekat dengan pengguna dalam konteks kualitas pengalaman. Selain itu, Anvari dan Lu (2021) menggambarkan *active probing* sebagai gangguan terkontrol yang terukur yang mengumpulkan informasi tentang kondisi jaringan melalui respons terhadap pencarian. Hal ini berguna dalam situasi di mana observasi pasif tidak dapat memberikan pemisahan kondisi yang cukup.

Secara konseptual, rangkaian tes aktif yang tersusun membantu mengaitkan gejala degradasi dengan lapisan layanan yang mungkin menjadi sumber masalah. Dengan demikian, *active probing* menyediakan sinyal yang cocok untuk sistem monitoring berbasis pengalaman yang memerlukan indikator terukur sebagai dasar pengambilan keputusan.

2.2.3 Event-driven / event-triggered monitoring

Monitoring periodik bekerja dengan melakukan *sampling* pada interval tetap. Kelemahan dari pendekatan monitoring konvensional tersebut adalah *event* transien berdurasi pendek dapat terlewat ketika jatuh di antara dua interval *sampling*, sementara interval yang makin rapat akan meningkatkan *overhead*. Pendekatan *event-driven/event-triggered monitoring* menawarkan alternatif untuk mengatasi kekurangan tersebut. Sistem ini mengamati kondisi secara kontinu, namun mengaktifkan tindakan khusus (alarm, perekaman detail, atau pengiriman event) hanya ketika kondisi pemicu terpenuhi.

Kučera et al. (2020) mengusulkan *monitoring push-based* yang mengirim notifikasi hanya bila diperlukan agar pemrosesan data yang tidak perlu dapat dihindari, sehingga *overhead* lebih terkendali dibandingkan dengan pendekatan *polling* murni. Pada skala *data plane*, Zhou et al. (2020) menunjukkan konsep *flow event telemetry* yang bertujuan menemukan dan merekam *event* kinerja penting dengan overhead kecil melalui pemrograman *data plane.*

2.2.4 Ring buffer dan pre/post window untuk perekaman bukti insiden

Salah satu tantangan utama dalam perekaman kejadian (*event recording*) adalah kebutuhan untuk menyimpan konteks sebelum suatu *event* terdeteksi (*pre-event context*). Hal ini menjadi penting karena keputusan bahwa suatu kondisi merupakan gangguan atau anomali umumnya baru dapat diambil setelah gejala tertentu muncul. Tanpa mekanisme penyimpanan konteks sebelumnya, informasi yang relevan untuk analisis penyebab (root cause analysis) dapat hilang.

Untuk mengatasi kebutuhan tersebut, sistem umumnya menggunakan mekanisme *ring buffer* atau *circular buffer*, yaitu struktur data yang mempertahankan jendela data berjalan

(*sliding window*) dengan kapasitas tetap. Dalam mekanisme ini, data terbaru akan secara otomatis menggantikan data terlama ketika kapasitas penuh, sehingga sistem selalu menyimpan rekaman data terkini dalam rentang waktu tertentu tanpa memerlukan alokasi memori tambahan secara dinamis (Marwedel, 2021).

2.2.5 Fault injection dan ground truth untuk evaluasi sistem deteksi

Evaluasi sistem deteksi *event* memerlukan data dengan label kebenaran (*ground truth*), terutama untuk menghitung metrik seperti *precision*, *recall*, dan *F1-score*, serta untuk mengukur ketepatan waktu deteksi (latensi deteksi). Tanpa keberadaan *ground truth*, sulit untuk menilai apakah sistem benar-benar mampu mendeteksi gangguan secara akurat dan tepat waktu, atau hanya menghasilkan alarm yang bersifat kebetulan.

Salah satu pendekatan yang umum digunakan untuk memperoleh *ground truth* adalah *fault injection*, yaitu teknik menciptakan gangguan secara terkontrol, terukur, dan dapat direplikasi. Dengan cara ini, respons sistem dapat diamati pada kondisi yang sudah diketahui karakteristiknya, sehingga evaluasi performa menjadi lebih objektif dan sistematis. Cotroneo et al. (2022) memperkenalkan ThorFI sebagai pendekatan *network fault injection as a service* pada lingkungan *virtual network cloud*, serta menekankan pentingnya mekanisme injeksi yang bersifat *non-intrusive* dan terisolasi agar pengujian ketahanan sistem tidak menimbulkan efek samping yang tidak diinginkan (Cotroneo et al., 2022).

BAB 3 METODOLOGI

3.1 Metode yang digunakan

Penelitian ini merupakan penelitian pengembangan sistem yang berfokus pada perancangan dan implementasi prototipe Micro-UXI berbasis Arduino Uno Q sebagai *network experience event recorder* dengan pendekatan *event-driven monitoring.* Metode penelitian ini menggabungkan dua aspek utama, yaitu: (1) perancangan dan implementasi prototipe, dan (2) evaluasi eksperimental terukur secara kuantitatif untuk menilai kinerja deteksi *event*, kelengkapan bukti (*evidence bundle*), dan *overhead* sumber daya.

Secara konsep, sistem Micro-UXI yang dikembangkan pada penelitian ini memiliki dua fungsi yang berjalan pada saat bersamaan:

1. Monitoring metrik pengalaman jaringan secara kontinu (status koneksi Wi-Fi, ping, DNS, HTTP, dsb.)
2. Perekaman bukti insiden secara *event-driven* (*pre-event, event, post-event*) saat terdeteksi adanya degradasi atau anomali.

Evaluasi dilakukan dengan membandingkan Micro-UXI secara *black box event-driven* dengan *baseline monitoring* periodik berbasis threshold statis. *Baseline* merepresentasikan praktik monitoring konvensional yang melakukan pengukuran pada interval tertentu dan menghasilkan metrik dan *alert* tanpa mekanisme perekaman bukti *pre/post event* yang terstruktur.

3.1.1 Tahapan penelitian

Tahapan penelitian disusun secara sistematis agar implementasi dan evaluasi berjalan konsisten. Alur tahapan penelitian divisualisasikan pada Gambar 3.1. Tahanpan ini meliputi:

1. Perancangan arsitektur sistem

   Tahap ini menetapkan rancangan Micro-UXI sebagai *blackbox event recorder* yang melakukan *monitoring* telemetri, deteksi *event*, penyimpanan *ring buffer* untuk perekaman *pre-event* dan *post-event*, mekanisme perekaman evidence bundle, serta pipeline pengiriman dan penyimpanan data pada server. Pada tahap ini juga ditetapkan definisi *event*, komponen bukti, kebijakan privasi/retensi, serta rancangan format *dataset*.

2. Implementasi prototipe

   Tahap implementasi mencakup pengembangan komponen pada sisi sensor (Arduino UNO Q) dan sisi server. Pada sisi sensor diimplementasikan modul pengukuran telemetri, deteksi *event*, *ring buffer*, perekaman *evidence bundle*, serta pengiriman telemetri dan *event*. Pada sisi server diimplementasikan modul kolektor data, penyimpanan, dan dashboard untuk pemantauan.

3. Verifikasi fungsional sistem

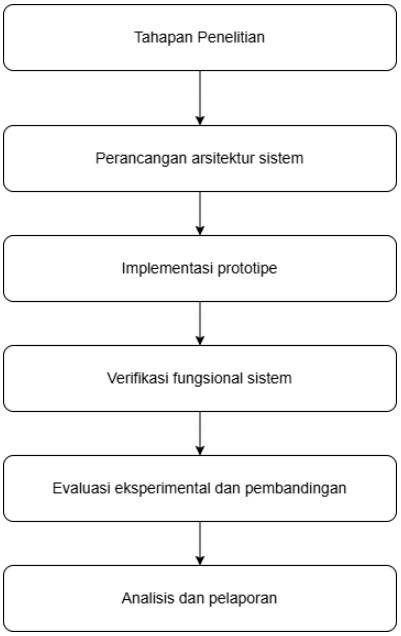
Tahap ini memastikan semua fungsi berjalan sesuai rancangan sebelum eksperimen dilakukan. Verifikasi mencakup konsistensi *timestamp*, kestabilan pengukuran telemetri, validasi aktivasi *trigger*, validasi *ring buffer* dan pembekuan *pre-event* dan *post-event*, validasi pembentukan *evidence bundle*, serta konektivitas dan pengiriman data ke *server*.

4. Evaluasi eksperimental dan pembandingan

   Tahap evaluasi dilakukan melalui eksperimen terkontrol dengan skenario *fault injection* yang memiliki parameter dan durasi terukur. Setiap skenario dijalankan pada Micro-UXI dan *baseline* pembanding. Seluruh *run* dicatat dengan identitas dan *timestamp* agar mendukung evaluasi menggunakan metrik yang ditetapkan.

5. Analisis dan pelaporan

   Pada tahap ini, data hasil eksperimen yang diperoleh dari seluruh skenario pengujian dikompilasi dan diproses secara sistematis untuk dianalisis berdasarkan metrik evaluasi yang telah ditetapkan.



Gambar 3.1 Diagram alur penelitian

3.1.2 Data yang diguanakan

Data penelitian diambil dari hasil operasi sistem Micro-Uxi dan hasil skenario uji yang terkontrol. Data terdiri dari:

Tabel 3.1 Data yang digunakan

Kategori	Deskripsi	Contoh data

Data Telemetri	Data pengukuran kontinu yang direkam selama sistem berjalan	wifi_link_state, rtt_ms, loss_pct, loss_pct, dns_latency_ms, dns_timeout, ram_mb, http_connect_ms
Data *ground truth* gangguan	Data yang dicatat pada saat proses *fault injection* pada laptop agar evaluasi menjadi objektif	run_id, scenario_id, fault_start_ts, fault_end_ts, parameter
Data *event*	Data *log* yang merupakan hasil dari deteksi sistem ketika *trigger* aktif	event_id, run_id, event_type, ts_start, ts_end, severity
Data *evidence bundle*	Paket data yang merupakan bukti dari suatu *event.*	event_meta.json, pre_window.csv, post_window.csv, net_snapshot.txt, wifi_snapshot.txt

3.2 Bahan dan peralatan yang digunakan

3.2.1 Perangkat keras

Secara operasional, Arduino Uno Q ditempatkan sebagai sensor yang terhubung dengan SSID uji. Laptop berperan sebagai pusat pengujian *fault injection* dan pengolahan data. Server digunakan sebagai penyimpanan data yang akan ditampilkan pada dashboard.

Tabel 3.2 Perangkat keras

Perangkat keras	Spesifikasi	Fungsi
Arduino UNO Q	MPU: Qualcomm Dragonwing QRB2210 RAM: 2 GB LPDDR4 *Connectivity*: Wi-Fi	Sensor
Laptop	Acer Nitro 5 AN-517	Pusat pengujian dan pengolahan data
Access Point / Wi-Fi	SSID: myITS Wi-Fi	Subjek uji
Server	Server DPTSI	Penyimpanan data untuk dashboard.

3.2.2 Perangkat lunak

Pada penelitian ini, data yang didapat diolah dan ditampilkan pada dashboard.

Tabel 3.3 Perangkat lunak

Perangkat	Lokasi	Fungsi
Program Micro-UXI	Arduino UNO Q	Mengukur telemetri
Program *fault injection*	Laptop	Menghasilkan gangguan terkontrol dan tercatat
Program analisis data	Laptop	Menghitung Precision/Recall/F1, FAR, MTTD, RCS, serta overhead
Dashboard	Server	Menyimpan dan menampilkan metrik

3.3 Perancangan arsitektur sistem

Arsitektur sistem Micro-UXI pada penelitian ini dirancang sebagai *black box event recorder.* Pada *monitoring* periodik biasa, sistem hanya berfokus pada merekam metrik dan menampilkan grafik pada dashboard. Pada sistem ini, arsitektur harus menambahkan mekanisme perekaman bukti lengkap (*pre-event, event, post-event*) ketika *trigger* aktif.

3.3.1 Arsitektur fungsional sistem

Arsitektur Micro-UXI yang dirancang terdiri dari enam bagian ini:

1. *Sensing layer*

   Probe menghasilkan telemetri (Wi-Fi/*link state*, RTT/*loss*, DNS, HTTP *timing*, dsb.)

2. *Baseline & detector*

   Telemetri digunakan untuk menentukan kondisi normal, lalu mendeteksi adanya deviasi signifikan menggunakan aturan yang ditentukan.

3. *Ring buffer*

   Probe menyimpan telemetri terbaru dalam *ring buffer* pada rentang waktu tertentu. Ketika *event* terdeteksi, *ring buffer* dibekukan untuk membentuk bagian *pre-event*.

4. *Event recorder*

   Ketika *trigger* aktif, *probe* akan menghentikan *ring buffer* sehingga bukti *pre-event* dan *post-event* tetap utuh. Kemudian *probe* akan merekan *evidence bundle* dan *snapshot* diagnostik secara lengkap hingga *event* selesai.

5. *Collector & storage*

   Server menerima telemetri dan menyimpan informasi terkait *event*.

6. *Dashboard*

   Dashboard menampilkan visualisasi metrik dan indikator status.

3.3.2 Alur kerja sistem dan model keadaan

Alur kerja sistem dirancang berbasis kondisi untuk memastikan perekaman bukti konsisten dan tidak tumpang tindih. Kondisi-kondisi tersebut adalah:

1. NORMAL: sistem melakukan sensing periodik, memperbarui *baseline*, dan mengisi *ring buffer.*
2. EVENT_ACTIVE: ketika kriteria *trigger* terpenuhi, sistem membekukan *ring buffer* sebagai *pre-event*, mulai merekam *event window*, serta mengambil snapshot *diagnostik* sesuai kebutuhan.
3. POST_EVENT/CLOSE: setelah kondisi pulih sesuai kriteria pemulihan (misalnya. metrik kembali ke rentang normal selama periode tertentu), sistem merekam *post-event*, menyelesaikan *evidence bundle*, memberi identitas *event*, dan menyiapkan pengiriman/penyimpanan.
4. BACK_TO_NORMAL: sistem kembali ke keadaan NORMAL dan melanjutkan *sensing*.

3.3.3 Definisi event dan parameter awal deteksi

Event didefinisikan sebagai kejadian degradasi yang signifikan dan memiliki dampak operasional bagi pengalaman pengguna. Jenis *event* yang digunakan pada penelitian ini ditetapkan agar selaras dengan skenario gangguan yang dapat diinjeksikan secara terkontrol. *Event-event* tersebut antara lain:

1. DNS_DEGRADED: latensi DNS meningkat melebihi ambang selama jendela waktu tertentu.
2. DNS_TIMEOUT_BURST: terjadi timeout DNS beruntun dalam periode singkat.
3. LOSS_BURST: terjadi lonjakan packet loss pada interval pendek.
4. HIGH_RTT: RTT meningkat signifikan dibanding *baseline* dalam periode tertentu.
5. HTTP_SLOW: waktu akses HTTP/TLS meningkat atau terjadi *timeout* pada *application check*.
6. CONNECTIVITY_FLAP: konektivitas terputus/tersambung berulang dalam waktu berdekatan.

3.3.4 Desain baseline dan aturan trigger

Baseline dibentuk dari telemetri *time-series* untuk menangkap pola normal dan mengurangi sensitivitas terhadap *outlier*. *Baseline* dapat dibangun menggunakan pendekatan *robust*, misalnya *rolling median* dan *Median* Absolute Deviation (MAD) pada *window* tertentu, atau metode *exponential smoothing* (EWMA). Untuk setiap metrik, dihitung ukuran deviasi (misalnya skor anomali) yang menjadi dasar *trigger*. Aturan *trigger* tersebut adalah:

1. Trigger aktif jika deviasi melebihi ambang selama N sampel berturut-turut (*burst logic*).
2. Trigger aktif jika terjadi perubahan keadaan (*state change logic*), misalnya, link Wi-Fi terputus.
3. Kriteria pemulihan (*recovery condition*) ditetapkan untuk menutup event, misalnya, metrik kembali ke rentang normal selama K detik.

3.3.5 Desain evidence bundle

Evidence bundle dirancang untuk memungkinkan rekonstruksi insiden dengan cakupan minimal sebagai berikut:

1. *Pre-event window*: telemetri pada interval sebelum *trigger* aktif (misalnya 30–60 detik).
2. *Event window*: telemetri selama *event* berlangsung.
3. *Post-event window*: telemetri setelah kondisi pulih (misalnya 30–60 detik).
4. *Snapshot* diagnostik jaringan: ringkasan status interface, konfigurasi IP/*routing*, dan indikator konektivitas.
5. *Snapshot* diagnostik Wi-Fi: ringkasan status Wi-Fi (BSSID, RSSI, bitrate, channel, jika tersedia).

3.4 Implementasi prototipe

Implementasi prototipe dilakukan pada sisi sensor (Arduino UNO Q) dan server.

3.4.1 Implementasi pada sensor

ada sisi sensor diimplementasikan:

1. *Telemetry probe*

   Melakukan pengukuran periodik untuk metrik jaringan dan aplikasi, seperti ping (RTT/*loss*), DNS *check* (*latensi*/*timeout*), HTTP *check* (*timing*), serta informasi link Wi-Fi (*status*, RSSI, *bitrate*, BSSID jika tersedia).

2. *Baseline updater* dan *detector*

   Mengolah sampel telemetri menjadi *baseline* dan mengevaluasi deviasi untuk menentukan *trigger*. *Detector* menghasilkan event_type, waktu deteksi, dan atribut pendukung (misalnya *severity*) jika event terjadi.

3. *Ring buffer*

   Menyimpan sampel telemetri terbaru. Ketika *trigger* aktif, *ring buffer* dibekukan sebagai *pre-event*.

4. *Evidence recorder*

   Mengelola perekaman *event window* dan *post-event window*, mengumpulkan *snapshot* diagnostik, menyusun *evidence bundle*, dan memberi struktur penyimpanan per *event*.

5. *Overhead monitor*

   Mencatat penggunaan CPU/RAM, pertumbuhan *storage*, serta volume data yang dikirim pada mode normal dan mode *event*.

6. *Data exporter*

   Mengirim telemetri/*event*/*evidence* ke *server* menggunakan protokol yang ditetapkan.

3.4.2 implementasi pada server

Pada sisi server diimplementasikan:

1. *Collector endpoint*

   Menerima telemetri, *event log*, dan *evidence bundle* dari *probe*.

2. *Storage manager*

   Menyimpan *dataset* secara terstruktur dan konsisten (per *run* dan per *event*). Penyimpanan dapat menggunakan struktur folder terstandarisasi atau basis data ringan sesuai kebutuhan

3. *Dashboard*

   Menyajikan ringkasan dan visualisasi telemetri dan *event* untuk membantu observasi dan validasi eksperimen.

3.5 Verifikasi fungsional sistem

Setiap fungsi sistem akan diverifikasi kerjanya sebelum dilakukan eksperimen. Beberapa verifikasi tersebut antara lain:

1. Validasi sinkronisasi waktu

   Memastikan *timestamp* pada *probe* dan perangkat *fault injection* berada dalam sinkronisasi.

2. Validasi *sampling* telemetri

   Memastikan seluruh metrik dapat diukur sesuai *interval* yang ditetapkan dan tidak terjadi kehilangan sampel yang sistematis.

3. Validasi *baseline* dan *trigger*

   Memastikan baseline terbentuk pada fase *warm-up* (awal sebelum *event*) dan *trigger* aktif ketika kondisi memenuhi kriteria.

4. Validasi *ring buffer* dan *evidence bundle*

   Memastikan data *pre-event* tersedia, *event window* dan *post-event* terekam sesuai durasi, dan *snapshot* diagnostik tersimpan lengkap.

5. Validasi *pipeline* pengiriman dan penyimpanan

   Memastikan telemetri, *event log*, dan *evidence bundle* dapat dikirim dan tersimpan dengan struktur *dataset* yang konsisten.

3.6 Evaluasi eksperimental dan pembandingan

Evaluasi dilakukan untuk membandingkan secara *black box event-driven* dengan *baseline monitoring* periodik berbasis *threshold* statis, menggunakan eksperimen yang terkontrol.

3.6.1 Desain run dan unit analisis

Satu kali *run* didefinisikan sebagai satu sesi eksperimen terstruktur yang terdiri dari:

1. *warm-up* baseline (kondisi normal),
2. fase *injection* (gangguan atau kondisi normal untuk uji FAR), dan
3. *cooldown* (kondisi normal setelah gangguan dihentikan).

Unit analisis meliputi:

1. *Fault*: gangguan yang diinjeksikan dengan waktu mulai/akhir dan parameter.
2. *Event*: output deteksi sistem yang memiliki rentang waktu dan tipe *event*.
3. *Evidence bundle*: paket bukti rekonstruksi untuk setiap *event*.

3.6.2 Lingkungan uji dan kontrol variabel

Testbed terdiri dari *probe* (UNO Q), AP/SSID uji, perangkat *fault injector*, serta *server* penyimpanan. Variabel kontrol ditetapkan untuk mengurangi variasi tak terkontrol yang meliputi: lokasi probe relatif terhadap AP, interval, profil target uji (domain DNS dan URL/*endpoint*), serta urutan pelaksanaan skenario yang diacak.

Jika penggunaan target eksternal berpotensi menimbulkan variasi yang sulit dikendalikan, *endpoint* lokal (DNS lokal dan/atau *web server* lokal) digunakan agar gangguan dapat diatur secara presisi.

3.6.3 Skenario uji (fault injection)

Skenario gangguan dirancang agar dapat diulang dan memiliki parameter terukur. Beberapa skenario yang digunakan adalah:

1. DNS *Delay*: penambahan *delay* pada trafik DNS (misalnya 200–800 ms) selama durasi tertentu.
2. DNS *Outage Burst*: pemutusan resolusi DNS (drop/timeout) secara *burst* dalam periode singkat yang berulang.
3. Packet *Loss Burst*: injeksi *packet loss* (misalnya 5–20%) pada interval pendek.
4. RTT *Increase*: penambahan delay RTT (misalnya 100–500 ms) dalam durasi menit.
5. *Bandwidth Throttling*: pembatasan *throughput* (misalnya 1–5 Mbps) selama durasi tertentu.
6. *Connectivity Flap*: pemutusan konektivitas secara berulang dengan durasi putus singkat.

Setiap skenario dijalankan dalam beberapa repetisi agar hasil statistik memadai. Parameter skenario dicatat dalam *log ground truth*.

3.6.4 Baseline pembanding

Baseline pembanding merepresentasikan *monitoring* konvensional berbasis pengukuran periodik dengan *threshold* statis, tanpa perekaman *evidence bundle* terstruktur. Dua baseline yang digunakan adalah:

1. *Ping-only monitoring*

   Melakukan ping periodik (misalnya setiap 60 detik) dan memicu alarm jika RTT/*loss* melampaui *threshold* statis.

2. *DNS + HTTP periodic check*

   Melakukan pemeriksaan DNS dan HTTP secara periodik (misalnya setiap 60 detik) dan memicu alarm jika *timeout*/latensi melampaui *threshold* statis. *Output baseline* berupa alarm log tanpa *evidence bundle pre/post-event*.

3.6.5 Prosedur uji per run

Prosedur uji dilakukan secara konsisten untuk seluruh skenario:

1. Inisialisasi sistem: memastikan *probe* terhubung, layanan Micro-UXI berjalan, sinkronisasi waktu aktif, serta konfigurasi target uji tersimpan.
2. *Warm-up baseline*: menjalankan kondisi normal selama durasi tertentu untuk membentuk baseline.
3. Aktivasi gangguan: menjalankan skrip *fault injection* dan mencatat fault_start_ts, fault_end_ts, scenario_id, serta parameter.
4. *Observasi* perekaman *event*: membiarkan Micro-UXI mendeteksi dan merekam evidence bundle.
5. *Cooldown*: menghentikan gangguan dan menjalankan kondisi normal untuk merekam *post-event*.
6. Ekspor *dataset*: menyimpan hasil *run* pada struktur *dataset* yang ditetapkan.
7. Repetisi: mengulang *run* sesuai jumlah repetisi dan mengacak urutan skenario untuk mengurangi bias.

3.6.6 Pencocokan event-fault (ground truth alignment)

Evaluasi dilakukan pada tingkat *event*. Setiap *fault* pada ground truth dipasangkan dengan event yang terdeteksi menggunakan toleransi waktu δ (misalnya 2–5 detik). Event dianggap mewakili fault apabila:

1. event_type sesuai dengan tipe *event* yang diharapkan pada skenario, dan
2. ts_start berada dalam jendela waktu [fault_start_ts − δ, fault_end_ts + δ]

Jika satu *fault* menghasilkan lebih dari satu *event*, strategi pencocokan ditetapkan sejak awal, misalnya *first-match* (*event* pertama yang memenuhi syarat) atau *best-match* (*event* dengan *overlap* terbesar). Aturan ini diperlukan agar perhitungan *precision/recall/F1* dan MTTD tidak ambigu.

3.6.7 Pengukuran overhead

Overhead diukur pada dua mode operasi:

1. Mode normal: sistem hanya melakukan *sensing* periodik dan pengiriman telemetri.
2. Mode *event*: sistem melakukan perekaman *evidence bundle* (*pre/event/post*), snapshot diagnostik, dan pengiriman data tambahan.

Metrik overhead yang dicatat mencakup:

1. CPU (rata-rata dan p95),
2. RAM (p95),
3. pertumbuhan storage (misalnya MB/run),
4. bandwidth telemetry dan evidence (misalignment MB/run).

3.7 Analisis data dan pelaporan

3.7.1 Pra-pemrosesan data

Tahap pra-pemrosesan meliputi:

1. sinkronisasi *timestamp* antarsumber data (*probe* dan *fault injector*),
2. normalisasi format data,
3. pembersihan data, dan
4. pelabelan *event* berdasarkan *ground truth* menggunakan aturan pencocokan *event–fault*.

3.7.2 Metrik evaluasi

Metrik evaluasi yang digunakan meliputi:

1. *Precision*, *Recall*, dan F1-*score* (*event-level*)
    a. TP: *fault* dipasangkan dengan event yang benar.
    b. FP: *event* muncul saat tidak ada *fault* atau tidak memenuhi aturan pencocokan.
    c. FN: *fault* terjadi, tetapi tidak ada *event* yang memenuhi aturan pencocokan.
    d. *Precision* = TP/(TP+FP)
    e. *Recall* = TP/(TP+FN)
    f. F1 = 2×(*Precision*×*Recall*)/(*Precision*+*1*)
2. *Mean Time To Detect (MTTD)*

    MTTD dihitung sebagai selisih waktu antara fault_start_ts dan waktu deteksi event (misalnya waktu ts_start event atau timestamp trigger pertama, sesuai definisi yang dikunci sejak awal).

3. *False Alarm Rate (FAR)*

    FAR dihitung sebagai jumlah FP per satuan waktu pada run kondisi normal (mis. event/jam), untuk mengukur kecenderungan sistem memunculkan alarm palsu.

4. *Reconstruction Completeness Score (RCS)*

    RCS digunakan untuk menilai kelengkapan evidence bundle. RCS disusun sebagai skor 0–1 berdasarkan ketersediaan komponen bukti minimal (pre-event, post-event, snapshot jaringan, snapshot Wi-Fi, dan bukti aplikasi bila digunakan). Skema penilaian menggunakan bobot yang konsisten sehingga dapat dibandingkan antarskenario.

5. *Overhead*

    Overhead dianalisis untuk menilai kelayakan operasi jangka panjang. Evaluasi dilakukan pada mode normal dan mode event, serta dilaporkan sebagai nilai rata-rata/p95.

3.8 Timeline penelitian

Penelitian ini dirancang untuk mengikuti *timeline* pada Tabel 3.3.

Tabel 3.3 Timeline penelitian

Kegiatan	Bulan 1				Bulan 2				Bulan 3				Bulan 4			
Perancangan arsitektur sistem																

Implementasi prototipe																
Verifikasi fungsional sistem																
Evaluasi eksperimental dan pembandingan																
Analisis dan pelaporan																

BAB 4 HASIL DAN PEMBAHASAN

BAB 5 KESIMPULAN DAN SARAN

DAFTAR PUSTAKA

LAMPIRAN

BIODATA PENULIS